Decision Trees

GENERAL ASSEMBLY

Agenda

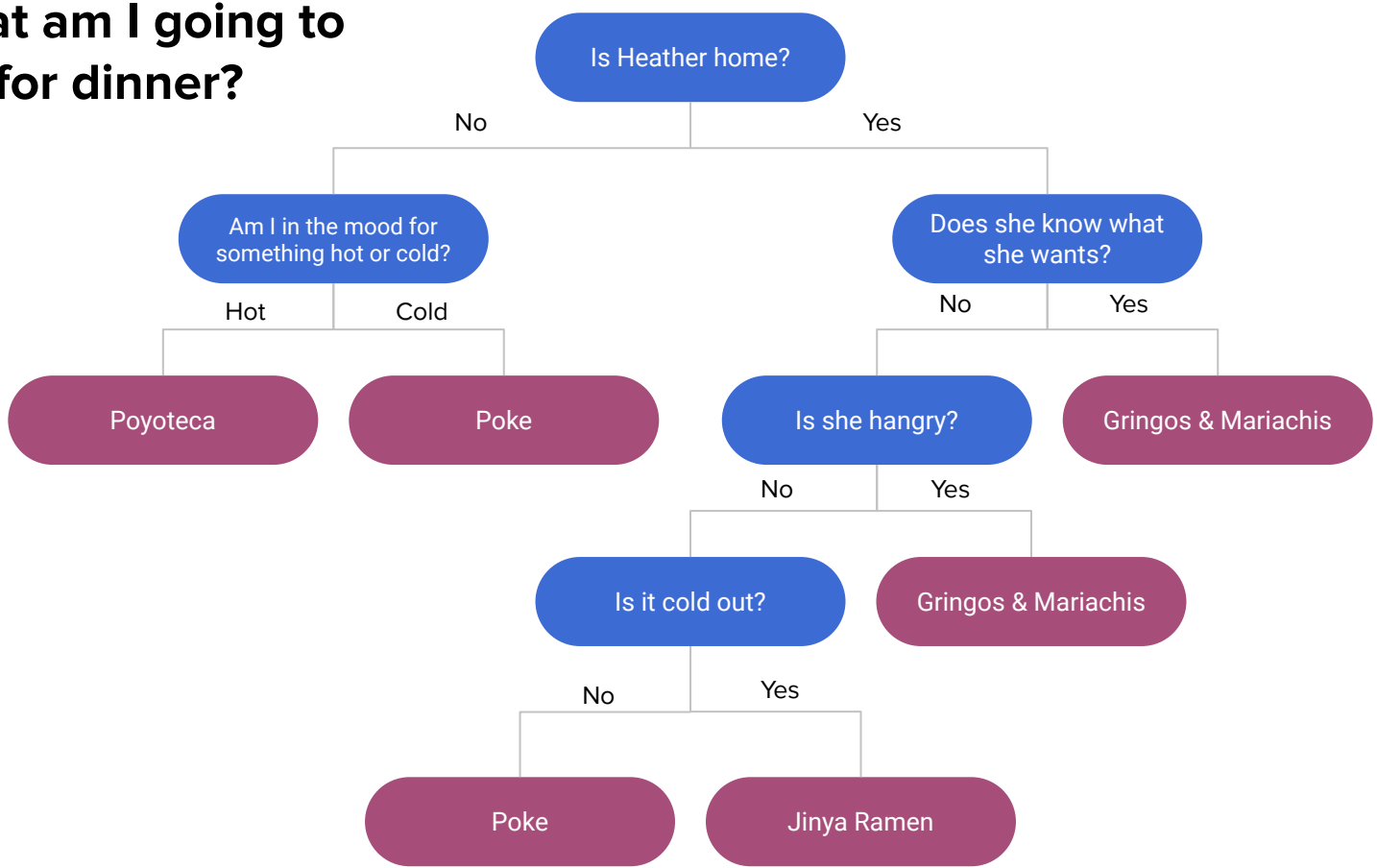
- What is a decision tree in general?
- What do they look like from a DS perspective?
- *Partitioning the feature space!*
- Stopping conditions and tuning parameters
- Python!

 © 2022 General Assembly

Decision Trees

We make decisions every day, often without even thinking about it. A **decision tree** is basically a flowchart that terminates with a decision. Each split in the flowchart represents some condition being true or false.

What am I going to eat for dinner?



 © 2022 General Assembly

Decision Tree Models

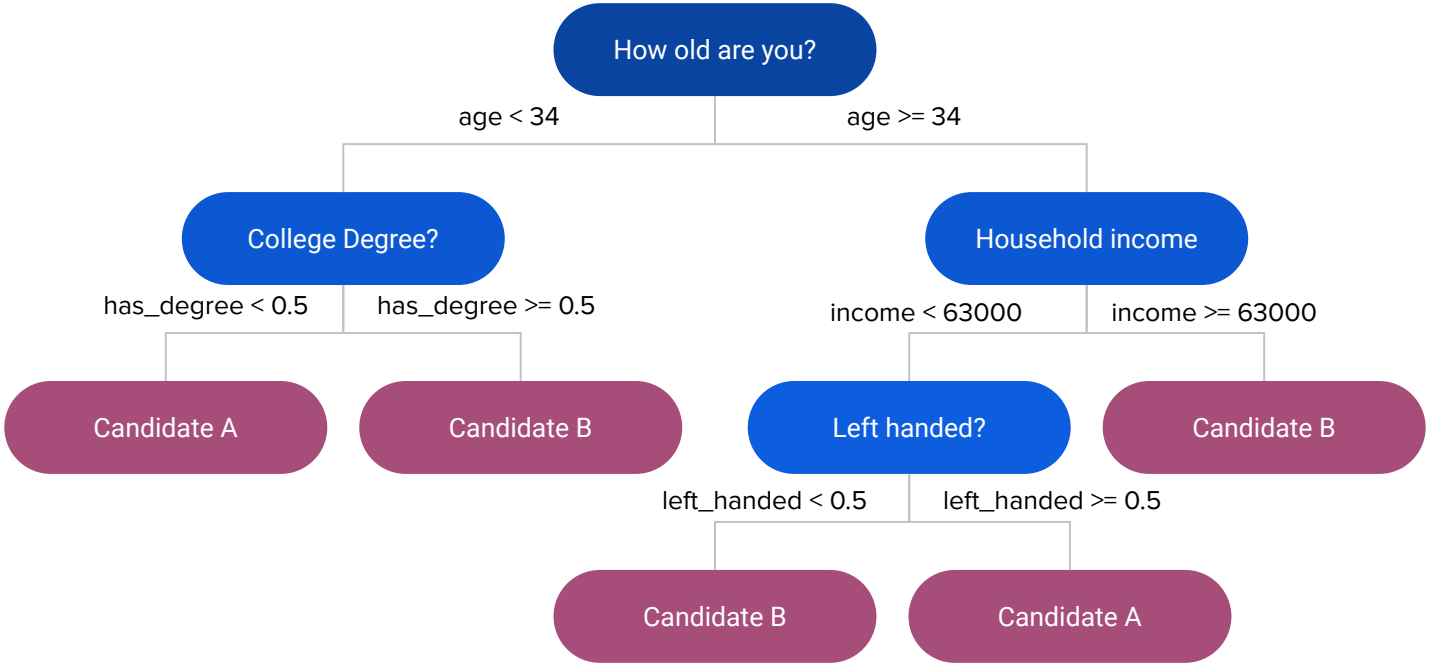
We'll be fitting decision trees as models, and so **their structure will be learned from our data**.

For linear models, an algorithm learns β from our data.

For tree-based models, an algorithm learns **when/where tree splits occur**.

 © 2022 General Assembly

Example: Who will a given person vote for?



 © 2022 General Assembly

The Decision Tree Algorithm

The CART Algorithm

So how is this sort of model actually fit? Most commonly, the **CART (Classification and Regression Trees) algorithm** is used. It works as follows:

The CART Algorithm

1. Find the “best” possible split value in each variable, across each variable based on some **splitting criterion** (more on this soon)
2. Split your data at this optimal split
3. Repeat steps 1-2 for each new subset of the data you’ve created until some **stopping criterion** is met

4. *(There’s an optional **pruning step**, but not by default!)*

 © 2022 General Assembly

When do we stop?

The **stopping criterion** for a decision tree is actually up to you! It can be considered a **tuning parameter** as well, actually.

By default, a decision tree will not stop splitting until each observation is in its own leaf! This is the perfectly overfit model, so **any imposed stopping criterion can be thought of a regularizing factor.**

Stopping Criteria allowed in Scikit-Learn

- **`max_depth`**: How many levels deep will we allow our tree to grow?
- **`min_samples_split`**: What is the minimum number of samples needed to split a node? If a split is not possible based on this criterion, we're done.
- **`min_samples_leaf`**: What is the minimum number of samples that we allow in a leaf node?
    - This is so similar to `min_samples_split` that we usually don't touch this.
- **`min_impurity_decrease`**: What's the minimum amount we need our stopping criterion to drop to allow a split? (We won't touch this one)

 © 2022 General Assembly

Representations

“Partitioning the Feature Space”

Decision trees are pretty and easy to read. For that we consider decision trees to be **white box models.**

However, there’s another representation of decision trees that we sometimes prefer to consider. We can think of these splits as

“partitioning the feature space”

“Partitioning the Feature Space”

Decision trees are pretty and easy to read. For that we consider decision trees to be **white box models.**

However, there’s another representation of decision trees that we sometimes prefer to consider. We can think of these splits as

“partitioning the feature space”

partitioning: separating into many parts

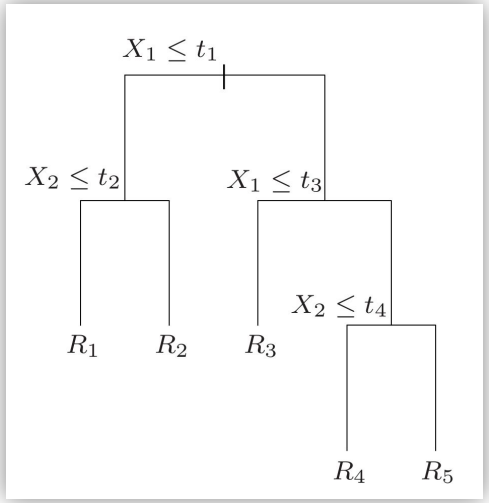
feature space: the range of all values our *xs* take

 © 2022 General Assembly

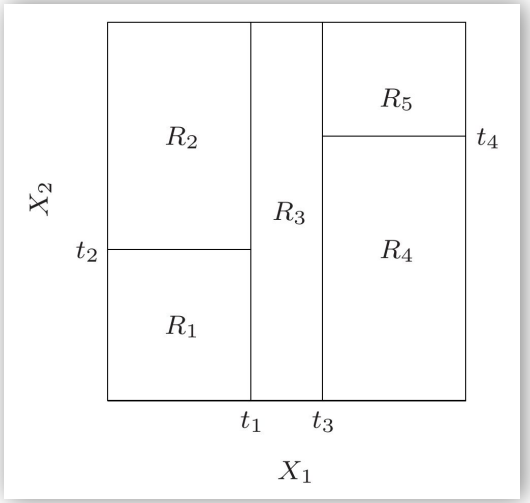
Let’s Draw a Picture



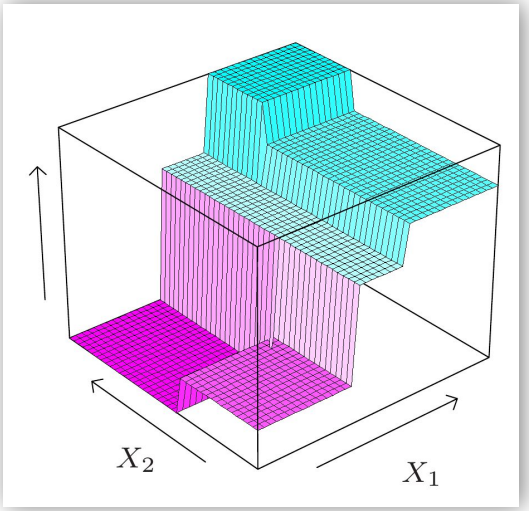
Three ways to draw the same thing!



Decision tree's tree representation



Decision tree's partitioning of the feature space representation



Same as the second image drawn in 3D for a regression problem

Images gleefully stolen from our holy text: Elements of Statistical Learning (ESL), by Hastie, Tibshirani, and Friedman

Splitting Criteria

Splitting Criteria

What makes one split better than another? The **splitting criteria** is the one that minimizes some **loss function** the most if the model were to stop right there.

For regression problems, this loss function is simply the **MSE**.

For classification, you might think we'd use accuracy or something, but in fact, we're going to learn a new one...

 © 2022 General Assembly

The Gini Impurity Index

The Gini index is a loss function used to describe the **purity** of the result of a classification problem.

Let p_i be the proportion of class *i* in a given leaf node. The **Gini impurity index** of that node is:

$$G = 1 - \sum p_i^2$$

 © 2022 General Assembly

The Gini Impurity Index Examples

$$G = 1 - \sum p_i^2$$

Consider a binary classification problem. In a **pure** node (ie, one that's totally one class), we have

$$G = 1 - \sum p_i^2 = 1 - 0^2 - 1^2 = 0$$

And in perfectly **impure** node where the classes are 50/50, we have

$$G = 1 - \sum p_i^2 = 1 - \left(\frac{1}{2}\right)^2 - \left(\frac{1}{2}\right)^2 = 1 - \frac{1}{4} - \frac{1}{4} = \frac{1}{2}$$

 © 2022 General Assembly

The Gini Impurity Index

The Gini index is actually pretty similar to its cousin **cross-entropy** (more on that later in the course) and shares its key benefit: differentiability.

However in practice the Gini index is preferred because of its preference for pure nodes.

 © 2022 General Assembly

Less yackin', more clackin'



 © 2022 General Assembly